

Client Side Technologies

HTML DOM

JS

Created by Ahmed Samir

HTML DOM



Standard for DOM Operations

Get, change, add, delete
HTML elements



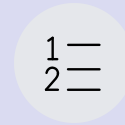
Hierarchy of Data Types

Documents, links, forms,
comments



Node-Based Structure

Objects with attributes
and children



Node Types

Represented by numeric
constants

Everything is a Node

The DOM organizes HTML content as a hierarchy of connected node types.

1

Document Node

Root of the DOM tree

2

Element Node

HTML tags and structure

3

Text Node

Content inside elements

4

Attribute Node

Properties on elements

5

Comment Node

Notes in the document

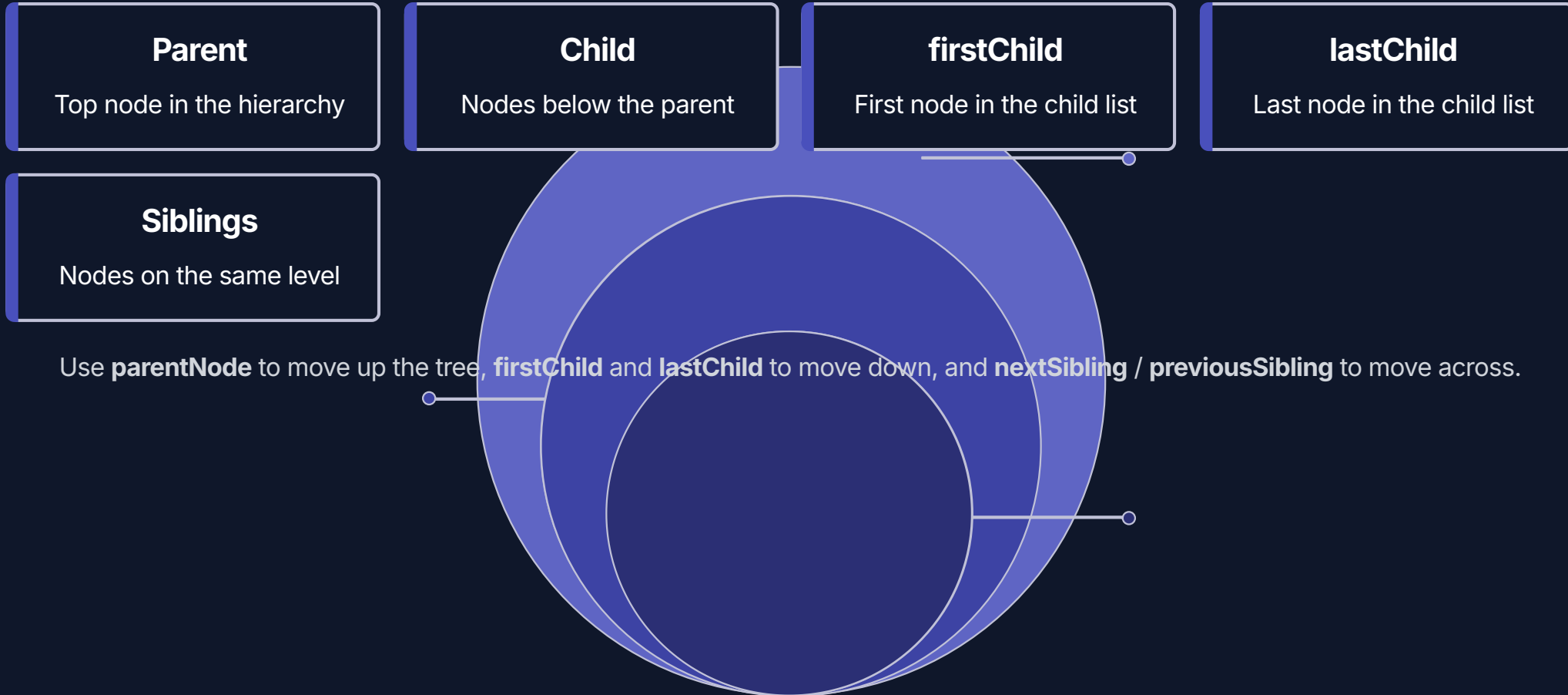
HTML Structure Example

```
<html>
  <head>
    <title>Simple Example!
  </head>
  <body>
    <h1>Heading
    <p id="my">A paragraph

    <!--and that's all-->
  </body>
</html>
```

Node Tree Relationships

The HTML DOM views an HTML document as a node-tree.



Node Relationships Explained

Parent Nodes

Have children nodes

Sibling Nodes

Same level, brothers or sisters

Child Nodes

Descendants of parent

Node Relationships in Practice

```
<html>
<head>
  <title>Simple Example!
</head>
<body>
  <h1>Greeting
  Welcome All
  <p>A paragraph

  <!--and that's all-->
</body>
</html>
```

```
#document
HTML
HEAD
TITLE
#text
BODY
H1
#text
P
#comment
```

Node Properties

All nodes have three main properties:

 **nodeName / tagName**

Returns HTML tag name in uppercase

 **nodeType**

Returns a numeric constant (12 node types)

 **nodeValue**

Returns null except for text and comment nodes

Root Element: document.documentElement

nodeName returns: #text (text), #comment (comment), #document (document)

Node Collections

Node collections help you work with groups of related nodes.

length property

Gives the count of items in the collection

Example: `childNodes.length` returns number of child elements

Check child collections

```
if (node.hasChildNodes()) {  
  console.log("Node has children");  
}
```

Use **hasChildNodes()** to check whether a node has any children.

Working with Nodes

01

Accessing Nodes

02

Modifying Node Content

03

Adding New Nodes

04

Removing Nodes from Tree

 JavaScript Yellow accents highlight each step in the workflow.

5 Ways to Access DOM Nodes

1

document.getElementById
("id")

2

document.getElementsByTagName
("tag")

3

document.getElementsByClassName
("class")

4

document.querySelector
("selector")

5

document.querySelectorAll
("selector")

Navigate the Node Tree

firstChild

Move directly to first child

lastChild

Move directly to last child

parentNode

Access the child's parent

nextSibling

Navigate down the tree one node step

previousSibling

Navigate up the tree one node step

childNodes[]

Using children collection

Modern Selectors: `querySelector()`

HTML5 new selectors let you target elements with CSS-style rules.

First `<div>` met

```
document.querySelector("div")
```

All `<div>` elements

```
document.querySelectorAll("div")
```

First with class `SomeClass`

```
document.querySelector(".SomeClass")
```

First with id `somelD`

```
document.querySelector("#somelD")
```

Modifying Node Content

innerHTML

Sets or returns HTML contents

innerText

Sets or returns text

textContent

Equivalent to innerText

nodeValue

Works with text and comment nodes

setAttribute()

Modifies/Adds attributes

element.attribute

Using attributes as properties

element.style.propertyName

Modifying styles

Creating & Adding Nodes

createElement()

Create a new tag element

createTextNode()

Create a new text element

createAttribute()

Create an attribute element

createComment()

Create a comment element

appendChild()

Add new node to DOM Tree at end

cloneNode(true|false)

Deep copy or shallow copy

insertBefore()

Insert before a specified element

Removing DOM Nodes

removeChild()

Remove a node from the DOM tree

replaceChild()

Remove a node and put another in its place

removeAttribute()

Remove a specified attribute

 **Quick tip:** To wipe all content of a subtree, set innerHTML to empty string: `document.body.innerHTML = "";`

addEventListener() Method



Attaches an event handler to the specified element



Does NOT overwrite existing event handlers



You can add many event handlers to one element



Multiple handlers of the same type are supported



Can be added to any DOM object, not just HTML elements



Makes it easier to control how the event reacts to bubbling



Separates JavaScript from HTML markup

addEventListener() Syntax & Parameters

SYNTAX

```
element.addEventListener(event, function, useCapture);
```

EXAMPLE

```
document.getElementById("b1").addEventListener("click",  
myFunction);
```

FUNCTION EXAMPLE

```
function myFunction() { alert("Button Clicked"); }
```

1st parameter

event type ("click", "mousedown", etc.)

2nd parameter

function to call when event occurs

3rd parameter

(optional): true = capturing phase, false = bubbling phase (default)

Removing Event Listeners

Use **removeEventListener()** to detach an event handler from an element.

removeEventListener() method:

```
element.removeEventListener("mousemove", myFunction);
```

⚠ Browser compatibility note: older browsers such as IE 8 and Opera 6.0 do not support **addEventListener()** / **removeEventListener()**.

Use **attachEvent()** / **detachEvent()** for those browsers:

```
element.attachEvent(event, function);
```

```
element.detachEvent(event, function);
```

HTML DOM Summary

Access Nodes

parentNode, childNodes

firstChild, lastChild

nextSibling,
previousSibling

getElementById()

getElementsByTagName()

getElementsByName()

HTML5 New Selectors

Modify Nodes

innerHTML or
innerText/textContent

nodeValue

setAttribute()

or attribute properties

Add Nodes

appendChild()

cloneNode()

insertBefore()

Remove Nodes

removeChild()

or replaceChild()

Thank You!

Client Side Technologies — HTML DOM



NO DESIGN